

Cơ sở dữ liệu thực hành

DML (tiếp)

Lê Thành Văn

Khoa Hệ thống Thông tin Quản lý

HUTECH

2026.05.18

1. Subquery
2. Common Table Expression (CTE)
3. TEMP TABLE
4. Bài tập

Cơ sở dữ liệu được sử dụng:

```
product_hierarchy(  
    id, parent_id, level_text, level_name  
)  
product_details(  
    product_id, price, product_name,  
    category_id, segment_id, style_id  
)  
sales(  
    product_id, qty, discount,  
    member, txn_id, txd_time  
)
```

Subquery

Subquery (truy vấn con) là một truy vấn được nhúng bên trong một truy vấn khác. Subquery thường được sử dụng để trích xuất dữ liệu từ một bảng hoặc nhiều bảng dựa trên kết quả của truy vấn chính.

Subquery có thể xuất hiện trong các mệnh đề SELECT, INSERT, UPDATE, DELETE và thậm chí trong một subquery khác.

Subquery luôn nằm trong dấu ngoặc đơn () và thường chạy trước để lấy kết quả cho câu lệnh chính sử dụng.

Một số phân loại subquery:

- Scalar: Truy vấn con đơn trị.
- Multiple-Row: Truy vấn con đa trị.
- Multiple-Column: Truy vấn con dạng bảng.
- Correlated: Truy vấn con tương quan.

Một số đặc điểm:

- Trả về đúng 1 ô duy nhất (1 hàng và 1 cột).
- Thường dùng với các toán tử $=$, $>$, $<$, $\geq$, $\leq$, $<>$.

Ví dụ 1

```
-- Truy vấn tên tất cả sản phẩm có segment_id giống
-- sản phẩm có tên 'Khaki Suit Jacket - Womens'
SELECT
    product_name
FROM product_details
WHERE segment_id = (
    SELECT segment_id
    FROM product_details
    WHERE product_name = 'Khaki Suit Jacket - Womens'
)
```

Một số đặc điểm:

- Trả về 1 cột nhưng có nhiều dòng dữ liệu (một danh sách các giá trị).
- Thường dùng với các toán tử **IN**, **NOT IN**.

```
-- Truy vấn tổng số sản phẩm đã bán của
-- tất cả mặt hàng có category_id = 1
SELECT
    SUM(qty)
FROM sales
WHERE product_id IN (
    SELECT product_id
    FROM product_details
    WHERE category_id = 1
)
```

Một số đặc điểm:

- Trả về nhiều cột và nhiều dòng dữ liệu (giống như một bảng).
- Thường dùng trong phần **FROM** như một bảng trung gian.

```
-- Truy vấn tên sản phẩm và số lượng bán ra của sản phẩm đó
SELECT
    pd.product_name, sc.sales_count
FROM product_details AS pd
    JOIN (
        SELECT
            product_id,
            SUM(qty) AS sales_count
        FROM sales
        GROUP BY product_id
    ) AS sc ON pd.product_id = sc.product_id
```

Một số đặc điểm:

- Câu lệnh con sử dụng dữ liệu từ câu lệnh chính. Do đó, nó không thể chạy độc lập mà phải chạy lặp đi lặp lại theo từng dòng của câu lệnh chính.
- Nếu chạy riêng câu lệnh trong ngoặc, PostgreSQL sẽ báo lỗi vì nó chứa các cột thuộc bảng của câu lệnh chính.

```
-- Truy vấn tên sản phẩm và số lượng bán ra của sản phẩm đó  
-- bằng cách dùng truy vấn tương quan
```

```
SELECT  
    (SELECT  
        product_name  
        FROM product_details AS pd  
        WHERE pd.product_id = sales.product_id),  
    SUM(qty) AS sales_count  
FROM sales  
GROUP BY product_id
```

EXISTS/NOT EXISTS là từ khóa của PostgreSQL dùng để kiểm tra sự tồn tại/không tồn tại của kết quả một câu truy vấn.


```
SELECT  
    EXISTS (  
        SELECT 1  
        FROM product_details  
        WHERE product_id = 20  
    )
```

Vì để kiểm tra sự tồn tại/không tồn tại nên câu truy vấn theo sau **EXISTS/NOT EXISTS** thường bắt đầu bằng **SELECT 1**.

```
-- Truy vấn tên mặt hàng được mua trong ngày 2021-02-13
SELECT
    product_name
FROM product_details AS pd
WHERE EXISTS (
    SELECT 1
    FROM sales AS s
    WHERE s.product_id = pd.product_id
        AND start_txn_time::date = '2021-02-13'
)
```

Subquery thường được dùng:

- Lọc dữ liệu dựa trên một giá trị 'chưa biết trước'.
- Gom nhóm và định hình lại dữ liệu.
- Tạo các cột dữ liệu động, tính toán theo từng dòng.
- Phục vụ cho các thao tác cập nhật hoặc xóa dữ liệu hàng loạt.

Một số hạn chế:

- Hiệu năng khi dùng truy vấn con tương quan.
- Sử dụng truy vấn con lồng nhau nhiều khiến câu lệnh phức tạp, khó theo dõi và khó bảo trì.
- Không có khả năng tái sử dụng

Vì các hạn chế trên, subquery chỉ được dùng khi cần truy vấn nhanh, các bảng dữ liệu nhỏ hoặc phục vụ việc viết báo cáo tức thời (ad-hoc query).

Common Table Expression (CTE)

Trong PostgreSQL, CTE (Common Table Expressions) là một tính năng cực kỳ mạnh mẽ, giúp gom một câu lệnh truy vấn phức tạp thành một 'bảng tạm thời' và đặt tên cho nó. Sau đó, có thể tái sử dụng bảng tạm này ngay trong câu lệnh **SELECT**, **INSERT**, **UPDATE** hoặc **DELETE** chính ở phía dưới.

```
;WITH tên_bảng_tạm AS (  
    -- Câu lệnh SQL định nghĩa dữ liệu (Subquery)  
    SELECT ...  
        FROM bảng_gốc  
        WHERE điều_kiện  
)  
-- Câu lệnh chính sử dụng bảng tạm ở trên  
SELECT ...  
    FROM tên_bảng_tạm  
    WHERE điều_kiện_tiếp_theo;
```


Ví dụ 7

```
-- Truy vấn tên sản phẩm và số lượng bán ra của sản phẩm đó
-- bằng cách dùng CTE
;WITH sales_count AS (
    SELECT
        product_id, SUM(qty) AS sales_count
    FROM sales
    GROUP BY product_id
)
SELECT
    pd.product_name, sc.sales_count
FROM product_details AS pd
    JOIN sales_count AS sc
    ON pd.product_id = sc.product_id
```

Một CTE chỉ tồn tại trong duy nhất một câu lệnh ngay sau khi nó được định nghĩa.

```
WITH bang_tam AS (  
    SELECT DISTINCT product_id FROM sales  
)  
SELECT * FROM bang_tam; -- Lệnh này chạy ĐÚNG  
  
-- Lệnh thứ hai này sẽ báo LỖI, do bang_tam không còn tồn tại.  
SELECT * FROM bang_tam;
```

Có thể đọc về những cách sử dụng khác của CTE (như tạo nhiều CTE cùng lúc, CTE đệ quy, ...) tại [trang tài liệu của PostgreSQL](#).

TEMP TABLE

Trong PostgreSQL, Temp Table (Bảng tạm thời) là một loại bảng đặc biệt được tạo ra để lưu trữ dữ liệu ngắn hạn. Nó cực kỳ hữu ích khi bạn phải xử lý một khối lượng dữ liệu lớn, cần chia nhỏ một bài toán tính toán phức tạp thành nhiều bước, hoặc muốn lưu lại kết quả trung gian để sử dụng cho nhiều câu lệnh SQL khác nhau chạy kế tiếp nhau.

Một số đặc điểm của TEMP TABLE:

- Tính cô lập (Session-scoped): Temp Table chỉ tồn tại trong session (phiên kết nối) tạo ra nó. Người khác kết nối vào cùng database đó sẽ không nhìn thấy và không thể sử dụng bảng tạm của bạn.
- Tự động xóa (Auto-drop): Khi bạn ngắt kết nối, postgres sẽ tự động xóa sổ bảng tạm này cùng toàn bộ dữ liệu bên trong nó.

Cú pháp tạo TEMP TABLE

```
-- Cách 1: Tạo bảng tạm trống và định nghĩa cột (giống bảng thường)
CREATE TEMP TABLE tên_bảng_tạm (
    định_nghĩa
);

-- Cách 2: Tạo bảng tạm từ kết quả của một câu lệnh SELECT
CREATE TEMP TABLE tên_bảng_tạm AS
    SELECT ... FROM ... WHERE ...;
```


Dù Temp Table tự xóa khi ngắt kết nối, vẫn nên dùng **DROP TABLE IF EXISTS** `ten_bang_tam`; ở cuối đoạn lệnh để tránh làm phình hệ thống.

Bài tập

Cho cơ sở dữ liệu:

- Nhân viên: `nhân_viên(mã_nv, họ_tên, phòng_ban, lương)`.
- Dự án: `dự_án(mã_da, mã_nv, tên_da, kinh_phí)`.

Thực hiện các câu truy vấn sau:

- 1 Hãy tìm thông tin (`họ_tên`, `lương`) của những nhân viên có mức lương thấp hơn mức lương của nhân viên tên 'Nguyễn Văn A' (giả sử `họ_tên` là duy nhất).
- 2 Hãy liệt kê danh sách các nhân viên (`họ_tên`, `phòng_ban`) đang phụ trách ít nhất một dự án.
- 3 Hãy tìm những nhân viên có mức lương cao hơn mức lương trung bình của chính phòng ban mà người đó đang làm việc (dùng subquery và cte).

-- câu 1

```
SELECT họ_tên, lương
      FROM nhân_viên
      WHERE lương < (
          SELECT lương FROM nhân_viên
            WHERE họ_tên = 'Nguyễn Văn A');
```

-- câu 2

```
SELECT họ_tên, phòng_ban
      FROM nhân_viên
      WHERE mã_nv IN (SELECT mã_nv FROM dự_án);
```

-- câu 3: dùng subquery

SELECT

họ_tên, nv.phòng_ban, lương

FROM nhân_viên AS nv

WHERE lương > (

SELECT AVG(lương)

FROM nhân_viên AS nv2

WHERE nv2.phòng_ban = nv.phòng_ban

);

```
-- câu 3: dùng cte
;WITH ltb AS (
    SELECT phòng_ban, AVG(lương) tb
    FROM nhân_viên
    GROUP BY phòng_ban
)
SELECT
    họ_tên, nv.phòng_ban, lương
FROM nhân_viên AS nv
    JOIN ltb ON nv.phòng_ban = ltb.phòng_ban
WHERE nv.lương > ltb.tb
```